

Problem 1 (10 marks)**Linear Independence, Singular Matrices, and Solutions of a SLE**

In data science and AI, we often represent information as vectors in a feature space. For example, suppose you are building a model where the columns of a matrix A represent different feature directions. If these features are linearly independent, then they provide unique and non-redundant information. If they are dependent, some features can be expressed as combinations of others, leading to redundancy.

Similarly, when we try to fit data (represented by a vector b) into this feature space, the system of linear equations $\mathcal{L}(A, b)$ corresponds to finding a combination of features that explains the data. Whether this system is consistent depends on whether b lies in the span of the columns of A .

- (a) Prove that if the column vectors of a square matrix A are linearly independent, then A is a non-singular (invertible) matrix.
- (b) Prove that the system of linear equations $\mathcal{L}(A, b)$ is consistent $\iff b$ belongs to the span of the columns of A .

Start with the **definitions**. What does it mean for the column vectors of A (let's call them $\vec{a}_1, \vec{a}_2, \dots, \vec{a}_n$) to be **linearly independent**? It concerns the solutions to the equation

$$x_1 \vec{a}_1 + x_2 \vec{a}_2 + \dots + x_n \vec{a}_n = \vec{0}.$$

A linear combination of the columns of A can be expressed as the matrix-vector product $A\vec{x}$. What does the linear independence of the columns tell you about the solution(s) to the homogeneous system $A\vec{x} = \vec{0}$? How does the solution set of $A\vec{x} = \vec{0}$ relate to the invertibility of A ? (Think about the null space or kernel of the matrix).

For part (b), this is an "if and only if" ($\iff$) proof, so you must prove two separate implications:

1. **Forward direction** ($\Rightarrow$): Assume (A, b) is consistent, and show this implies $\vec{b}$ is in the span of the columns of A .
2. **Backward direction** ($\Leftarrow$): Assume $\vec{b}$ is in the span of the columns of A , and show this implies (A, b) is consistent.

The entire proof hinges on one crucial idea: the interpretation of the matrix-vector product $A\vec{x}$. If A has columns $\vec{a}_1, \dots, \vec{a}_n$ and $\vec{x} = (x_1, \dots, x_n)^T$, then the product $A\vec{x}$ is precisely the **linear combination**:

$$A\vec{x} = x_1 \vec{a}_1 + x_2 \vec{a}_2 + \dots + x_n \vec{a}_n.$$

Problem 2 (10 marks)**Vector Spaces, Subspaces, and Dimension**

In some physical or sensing systems the response to a scalar stimulus differs by direction: one component responds linearly (keeping sign), while another responds only to the magnitude of the stimulus (losing sign information). Consider the set $V = \mathbb{R}^2$ with the usual vector addition $+$ but with a modified scalar multiplication

$$\alpha * \begin{pmatrix} x \\ y \end{pmatrix} := \begin{pmatrix} \alpha x \\ |\alpha| y \end{pmatrix} \quad (\alpha \in \mathbb{R}, (x, y) \in V).$$

This models an anisotropic scaling that discards the sign of the scalar on the second coordinate.

- (a) Prove that $(V, +, *)$ satisfies all axioms of a vector space but one. State which one and provide an example.

Start by listing all the axioms of a vector space. Since vector addition is standard, you can probably assume the axioms involving only addition will hold. The unusual part of this problem is the scalar multiplication, so focus your attention on the axioms that involve the multiplication.

To show an axiom fails, you just need a single counterexample. For the axiom you suspect is the problem, try picking specific numerical values for the scalars and the vector. Choose scalars that will highlight the unique behavior of the absolute value function.

Problem 3 (10 marks)

Gram-Schmidt

We often want to simplify vector computations by working with orthogonal or orthonormal sets of vectors. For example, in Principal Component Analysis (PCA) and other dimensionality-reduction techniques, we want to re-express data in a basis where the directions are orthogonal. This makes projections easy to compute and interpretations cleaner.

However, most sets of vectors we encounter are not orthogonal to begin with. The Gram-Schmidt process helps us systematically transform a set of linearly independent vectors into an orthogonal (or orthonormal) set spanning the same subspace.

Consider the vectors

$$v_1 = \begin{pmatrix} -7 \\ -i \\ 0 \\ 1 \end{pmatrix}, \quad v_2 = \begin{pmatrix} 2i \\ 3 \\ -1 \\ i \end{pmatrix} \in \mathbb{C}^4.$$

- (a) In practice, data features are rarely orthogonal. Apply the Gram-Schmidt procedure to $\{v_1, v_2\}$ to produce an orthogonal set. (This demonstrates how we can start with arbitrary features and produce ones that are easier to use for projections and decompositions.)
- (b) In high-dimensional spaces, we often want a full orthonormal basis to represent any possible vector. After orthogonalizing v_1 and v_2 , determine whether it is possible to extend your orthonormal set to a full basis for $\mathbb{C}^4$. If so, construct such a basis. (This connects to building complete coordinate systems, e.g. for QR factorization in machine learning.)
- (c) Subspaces defined by orthogonality conditions play a central role in machine learning, for example in defining null spaces and constraint spaces. Consider

$$W = \{v \in \mathbb{C}^4 \mid \langle v, v_1 \rangle = 0\}.$$

Determine whether W is a subspace of $\mathbb{C}^4$. (This shows how orthogonality naturally defines whole subspaces of valid solutions or constraints.)

Before you begin, remember that you are working in a **complex vector space** $\mathbb{C}^4$. The standard dot product from $\mathbb{R}^n$ (i.e., $u^T v$) is not sufficient here. We need a way to define the length of a vector that is always a non-negative real number.

This requires the **Hermitian inner product**, defined as:

$$\langle u, v \rangle = \bar{u}^T v = \sum_{k=1}^4 \bar{u}_k v_k$$

The crucial difference is the **complex conjugate** on the components of the first vector. This ensures that the inner product of a vector with itself, $\langle v, v \rangle = \sum |v_k|^2$, is always a real, non-negative number, which allows us to define a valid norm $\|v\| = \sqrt{\langle v, v \rangle}$. All calculations involving orthogonality and projections **must** use this inner product.

For part (b), you have two orthogonal vectors that span a 2D subspace of $\mathbb{C}^4$. To extend this to a basis for the full 4D space, you need to find two more vectors, let's call them w_3 and w_4 , that are in the **orthogonal complement** of the subspace you already have. This means you are looking for vectors w such that $\langle w, v_1 \rangle = 0$ and $\langle w, v_2 \rangle = 0$. The vectors you will find form a basis for the orthogonal complement, but they are not necessarily orthogonal to each other. What procedure can you apply to the them to make them orthogonal? (You've already used it in part a!)

Problem 4 (10 marks)**Chebyshev Inequality (Bounding Outliers in Data)**

In machine learning and data analysis, we often work with high-dimensional vectors representing samples, features, or signals. A useful way to summarize the overall magnitude of a vector is its *root mean square (RMS) value*, which measures the average energy of the components.

The Chebyshev inequality gives us a way to bound the number of “large” entries in a vector: if the RMS is small, not many entries can be very large in magnitude. This is directly connected to detecting outliers in datasets and understanding variance in features.

- (a) Suppose x is a 100-dimensional vector with $\text{rms}(x) = 1$. What is the maximum number of entries of x that can satisfy $|x_i| \geq 3$?

If your answer is k , explain why no such vector can have $k + 1$ entries with absolute values at least 3. Then construct an explicit example of a 100-vector with RMS value 1 that achieves exactly k entries larger than 3 in absolute value.

(Interpretation: This part asks you to quantify how many extreme outliers are even possible, given the average energy of the vector.)

- (b) Show that at least one entry of a vector must have absolute value at least as large as the RMS value of the vector.

(Interpretation: This guarantees the existence of at least one entry that is “representative” of the overall energy — no vector can have every entry below the RMS in magnitude.)

Start with the definition of the root mean square (RMS). You are given that $\text{rms}(x) = 1$ and the dimension is $n = 100$. Square the RMS definition and plug in the known values to find the required value of the sum of squares, $\sum_{i=1}^{100} x_i^2$. Think of this sum as your total “budget”. Suppose k entries of the vector satisfy $|x_i| \geq 3$. What is the *minimum* possible value for x_i^2 for each of these entries? Based on this, what is the *minimum* amount these k entries must contribute to the total sum of squares? This contribution cannot exceed the total budget you found, which gives you an inequality to solve for k .

To construct your example vector, aim to be as “efficient” as possible with your budget.

- Set your k large entries to have the smallest possible magnitude that still satisfies the condition (i.e., $|x_i| = 3$).
- Calculate how much of your sum-of-squares budget is used by these k entries.
- Distribute the remaining budget among the other $100 - k$ entries in the simplest way possible (many of them can be zero!).

For part (b), one possible method is a proof by **contradiction**. Start by assuming the opposite of what you want to prove. That is, assume that for *every* entry x_i , its absolute value is strictly less than the RMS value:

$$|x_i| < \text{rms}(x) \quad \text{for all } i = 1, \dots, n.$$

If $|x_i| < \text{rms}(x)$, what inequality can you write relating x_i^2 and $(\text{rms}(x))^2$? Apply this inequality to every term in the sum of squares $\sum_{i=1}^n x_i^2$ to find a strict upper bound for the sum.

From the definition of RMS, we know that $(\text{rms}(x))^2 = \frac{1}{n} \sum_{i=1}^n x_i^2$. Rearrange this to express the sum $\sum_{i=1}^n x_i^2$ in terms of n and $(\text{rms}(x))^2$. Now, compare this exact expression with the strict upper bound you found in. You should arrive at a statement that is logically impossible (like $A < A$), which proves your initial assumption was false.

Problem 5 (15 marks)

A hospital deploys a logistic regression model to estimate the probability that incoming patients have a serious condition (“positive”). Clinical decisions have asymmetric costs: missing a true positive (false negative) is typically much more costly than a false alarm (false positive). Thus, choosing a decision threshold matters; so does understanding precision/recall tradeoffs and the ROC/AUC.

You are given predicted probabilities from a trained logistic model and the true labels for 10 patients:

patient	1	2	3	4	5	6	7	8	9	10
$\hat{p}$	0.95	0.89	0.76	0.65	0.55	0.45	0.35	0.25	0.15	0.05
y	1	1	1	0	1	0	0	1	0	0

(There are five true positives and five true negatives.)

- (a) **(Threshold 0.5 — metrics).** Using threshold $t = 0.5$ (predict positive when $\hat{p} \geq 0.5$), compute:

- the confusion matrix (TP, FP, FN, TN),
- accuracy, precision, recall (sensitivity), specificity, and F1 score.

Why: shows how thresholding maps probabilities to discrete decisions and how different metrics measure different errors.

- (b) **(Cost-sensitive decision).** Suppose the clinical cost of a false negative is $C_{\text{FN}} = 10$ and the cost of a false positive is $C_{\text{FP}} = 1$ (correct predictions cost 0).

- Derive the optimal decision rule (threshold in terms of C_{FP} and C_{FN}) that minimizes expected cost for a single patient given predicted probability $\hat{p}$.
- Using the derived threshold, classify the 10 patients and compute the resulting confusion matrix and total expected cost (sum of realized misclassification costs).

Why: connects probabilistic outputs to operational decisions under asymmetric costs.

- (c) **(ROC & AUC).** Construct the ROC curve points by sweeping the threshold through the distinct predicted probabilities (include thresholds just above each distinct $\hat{p}$ and just below), compute the true positive rate (TPR) and false positive rate (FPR) at each step, and approximate the Area Under the Curve (AUC) using the trapezoidal rule. Report the AUC numeric value.

Why: AUC summarizes rank quality of the probabilistic model across thresholds.

- (d) **(Logistic formulation).** For a general binary dataset $\{(x_i, y_i)\}_{i=1}^n$ with $x_i \in \mathbb{R}^d$ and logistic model $P(y_i = 1 | x_i) = \sigma(w^\top x_i)$ where $\sigma(s) = 1/(1 + e^{-s})$:

- write the negative log-likelihood $L(w)$ (sum over examples),
- derive the gradient $\nabla L(w)$ and the Hessian $H(w)$ in matrix form,

Why: connects the loss function used to train logistic regression with the practical second-order optimization algorithm commonly taught for numerical fitting.

For part (a), apply the threshold $t = 0.5$ to get a binary prediction for each of the 10 patients. If a patient’s predicted probability $\hat{p}$ is 0.5 or greater, your prediction is 1 (positive); otherwise, it’s 0 (negative).

Compare your list of 10 predictions to the 10 true labels (y) to populate the confusion matrix. Systematically count the number of true positives (TP), false positives (FP), false negatives (FN), and true negatives (TN).

Once you have the confusion matrix values, the rest is direct application of formulas.

For part (c), ROC curve visualizes the trade-off between the model's ability to correctly identify positives (True Positive Rate) and its tendency to raise false alarms (False Positive Rate). You will plot points of the form (FPR, TPR). The key definitions are:

$$\text{TPR} = \frac{\text{TP}}{\text{Total Actual Positives}}, \quad \text{FPR} = \frac{\text{FP}}{\text{Total Actual Negatives}}$$

In this problem, the total number of actual positives is 5, and the total number of actual negatives is 5.

The smoothest way to generate the ROC points is to sweep a threshold from high to low.

- **Sort:** First, sort all patients by their predicted probability $\hat{p}$ in descending order (the problem already gives them in this order).
- **Start:** Begin with a threshold above the highest score (e.g., $t = 1.0$). At this point, you classify every patient as negative. Calculate your TP and FP (they will both be 0). This gives your starting ROC point: $(\text{FPR} = 0/5, \text{TPR} = 0/5) = (0, 0)$.
- **Iterate:** Lower your threshold one patient at a time, moving down your sorted list. After you cross each patient's score, they (and everyone with a higher score) are now predicted as positive. At each step, re-calculate your TP and FP counts based on this new group of predicted positives, and plot the new (FPR, TPR) point.
- **End:** Your final point will correspond to a threshold of 0, where everyone is classified as positive, resulting in the point $(1, 1)$.

The Area Under the Curve (AUC) can be found using the **trapezoidal rule**. Your ROC curve is a series of points connected by lines. The area underneath is a sum of rectangles and triangles, which can be calculated as a sum of trapezoids.

For each segment of your curve that moves from a point (x_1, y_1) to (x_2, y_2) , the area of the trapezoid underneath it is:

$$\text{Area of segment} = (\text{width}) \times (\text{average height}) = (x_2 - x_1) \times \frac{y_1 + y_2}{2}$$

Note that the width $(x_2 - x_1)$ is only non-zero when you encounter a **negative** example, which increases the FPR. Sum the areas of these trapezoids to get the total AUC.